

PROJECT REPORT

EMOTION AND SENTIMENT ANALYSIS USING NLTK

Submitted By

KUMARI SAUMYA

Registration No.: 25MIM10186

Project Domain	Natural Language Processing (NLP)
Programming Language	Python
Primary Library	NLTK
Visualization	Matplotlib

A detailed documentation of the project implementation, methodology, input, emotion mapping, sentiment analysis, visualization, results, limitations, and future scope.

1. ABSTRACT

This project implements an **Emotion and Sentiment Analysis** system using Python and Natural Language Processing techniques. The system reads a text document, preprocesses the content, identifies predefined emotional words, counts their occurrence, and performs sentiment analysis using the VADER sentiment analyzer available through NLTK. A Matplotlib bar chart is generated to visually represent the frequency of the detected emotions. The project provides a simple and understandable demonstration of how textual information can be converted into meaningful emotional and sentiment-related results.

2. INTRODUCTION

Natural Language Processing (NLP) is a field of artificial intelligence concerned with processing and understanding human language. Written text often contains information about a person's feelings, opinions, and emotional state. Emotion and sentiment analysis attempts to extract this information from textual data. In this project, a rule/mapping-based emotion detection approach is combined with VADER sentiment analysis. The implementation is intentionally simple so that the complete workflow—from raw text to visualization—can be understood and demonstrated clearly.

3. PROBLEM STATEMENT

The project addresses the problem of analyzing textual information to determine the emotions represented by words and the overall sentiment of the text. A system is required to take text as input, clean and normalize it, identify emotion-related words from a predefined mapping, calculate the frequency of each detected emotion, and determine whether the overall sentiment is positive, negative, or neutral.

4. AIM

To develop a Python-based NLP application that detects predefined emotions from textual data and performs overall sentiment analysis using NLTK's VADER Sentiment Analyzer, followed by graphical visualization of detected emotion frequencies.

5. OBJECTIVES

- **Text preprocessing:** Prepare raw text for analysis by converting it to lowercase and removing punctuation.
- **Tokenization:** Divide the text into individual words.
- **Stop-word removal:** Remove common English words that are not useful for the emotion-matching step.
- **Lemmatization:** Reduce words to their base form where applicable.
- **Emotion detection:** Match processed words with entries in the emotion mapping file.
- **Frequency analysis:** Count how often each emotion is detected.
- **Sentiment analysis:** Calculate VADER polarity scores and determine an overall verdict.
- **Visualization:** Display emotion frequencies using a bar graph.

6. SCOPE OF THE PROJECT

The project demonstrates a basic NLP pipeline for a text file and can serve as a foundation for larger text-analysis applications. Its present scope includes English text, predefined emotion mappings, VADER sentiment analysis, and a static Matplotlib visualization. It can be extended later to support larger vocabularies, additional text sources, user interfaces, or machine-learning-based classification.

7. TECHNOLOGIES AND TOOLS

Python: Used to implement the complete application.

NLTK: Provides tokenization, English stop words, WordNet lemmatization, and VADER sentiment analysis.

Matplotlib: Generates the emotion-frequency bar chart.

Counter: Counts occurrences of detected emotions.

Text Files: *read.txt* stores the input and *emotion.txt* stores word-to-emotion mappings.

PyCharm: Suitable development environment for running and testing the Python program.

8. SYSTEM REQUIREMENTS

Software: Python 3.x, PyCharm or another Python IDE, NLTK, Matplotlib.

Hardware: A standard computer capable of running Python applications is sufficient. The project does not require specialized hardware.

9. PROJECT FILE STRUCTURE

The recommended GitHub project structure is:

Emotion-Sentiment-Analysis/

■■■ main.py

■■■ read.txt

■■■ emotion.txt

■■■ graph.png

■■■ screenshots/

■ ■■■ output.png

■■■ README.md

■■■ STATEMENT.md

10. INPUT DATA

The project uses **read.txt** as its sample input. The provided input is:

"I am feeling so happy, glad, and excited today! However, yesterday I felt sad, angry, and fearful about the project."

The emotion dictionary contains mappings including *happy:happy*, *glad:happy*, *excited:happy*, *sad:sad*, *angry:anger*, and *fearful:fear*.

11. EMOTION MAPPING

The file **emotion.txt** stores one word-emotion pair per line. For example:

happy:happy

glad:happy

excited:happy

sad:sad

angry:anger

fearful:fear

During analysis, each mapping is checked against the lemmatized words obtained from the input text. When a match is found, the corresponding emotion is added to the detected-emotion list.

12. SYSTEM WORKFLOW

The complete workflow is:

Input Text → Lowercase Conversion → Punctuation Removal → Tokenization → Stop-word Removal → Lemmatization → Emotion Mapping → Emotion Frequency Counting → VADER Sentiment Analysis → Graph Generation.

This sequence separates text preparation from emotion detection and sentiment analysis, making the program easy to understand and modify.

13. DETAILED METHODOLOGY

Step 1 – NLTK resources: The program attempts to download the required resources: punkt, punkt_tab, stopwords, wordnet, and vader_lexicon.

Step 2 – File handling: The program checks whether the required input files exist. If they do not exist, fallback sample content can be created.

Step 3 – Preprocessing: The input is converted to lowercase and punctuation is removed. Tokenization converts the text into individual words. English stop words are filtered out, and the remaining words are lemmatized.

Step 4 – Emotion detection: The program reads the emotion mapping file and checks whether mapped words occur in the processed word list.

Step 5 – Frequency calculation: Python's Counter is used to determine the number of occurrences for every detected emotion.

Step 6 – VADER analysis: SentimentIntensityAnalyzer calculates negative, neutral, positive, and compound polarity scores. The compound score is used for the overall verdict.

Step 7 – Visualization: When matching emotions are available, Matplotlib creates a bar graph and saves it as **graph.png**.

14. ALGORITHM

1. Start the program.
2. Load required Python and NLTK libraries.
3. Download/check required NLTK resources.
4. Read the input text from read.txt.
5. Convert text to lowercase and remove punctuation.
6. Tokenize the cleaned text.
7. Remove English stop words.
8. Lemmatize the remaining words.
9. Read word-emotion mappings from emotion.txt.
10. Match mapped words with processed words.
11. Count detected emotions.
12. Calculate VADER polarity scores.

13. Print the overall sentiment verdict.
14. Generate and save the emotion-frequency graph.
15. End.

15. IMPLEMENTATION DETAILS

The main program imports `os`, `string`, `Counter`, `matplotlib.pyplot`, and NLTK modules. The text-processing section uses `word_tokenize`, English stop words, and `WordNetLemmatizer`. Emotion matching is performed by reading each line of the mapping file and comparing the mapped word with the lemmatized word list. The VADER function obtains polarity scores and uses the compound score thresholds to classify sentiment. Finally, the detected emotion frequencies are plotted using a Matplotlib bar chart.

16. OUTPUT AND RESULT ANALYSIS

The supplied output screenshot shows a Matplotlib bar chart with four detected emotion categories: **happy**, **sad**, **fear**, and **anger**. The chart shows that **happy** has a frequency of **2**, while **sad**, **fear**, and **anger** each have a frequency of **1**. This visualization confirms that the program detected multiple emotion categories and successfully converted the frequency information into a graphical result.

17. OUTPUT SCREENSHOT

The following is the actual output screenshot supplied for this project. It displays the generated emotion-frequency graph.

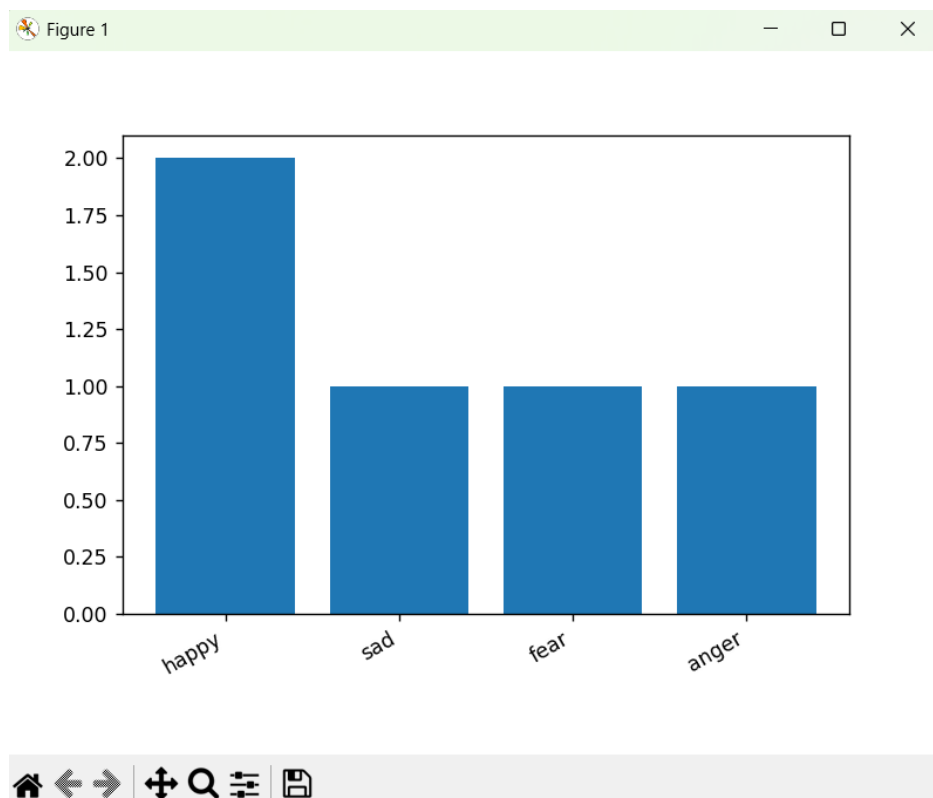


Figure 1: Actual emotion-frequency visualization generated by the project.

18. EXPECTED CONSOLE OUTPUT

When the program is executed, the console is designed to display the detected emotions list, emotion frequencies, VADER polarity scores, and an overall sentiment verdict. The exact VADER numerical values depend on the input text and the version/resources available in the environment. The emotion-frequency graph is saved as **graph.png**.

19. ADVANTAGES

- Easy to understand and implement.
- Demonstrates a complete basic NLP workflow.
- Uses an editable emotion dictionary.
- Combines emotion detection with sentiment analysis.
- Produces a visual result in addition to console output.
- Can be extended without redesigning the entire program.

20. LIMITATIONS

- Emotion detection depends on the words defined in emotion.txt.
- Words absent from the mapping may not be detected as emotions.
- The current implementation is not a trained machine-learning emotion classifier.
- The emotion mapping does not by itself capture complex context, sarcasm, or nuanced language.
- The visualization is a simple static bar chart.

21. FUTURE SCOPE

The system can be expanded with a larger and more detailed emotion lexicon, machine-learning or deep-learning classification, multilingual processing, direct user input, a graphical user interface, interactive visualization, database storage, and batch processing of multiple documents. These extensions could make the application more suitable for larger NLP experiments and practical text-analysis workflows.

22. APPLICATIONS

The underlying approach can be demonstrated in applications such as basic feedback analysis, educational NLP demonstrations, opinion analysis, text categorization experiments, and exploratory analysis of written comments. Any real-world deployment would require broader language coverage and more robust validation than the present rule-based prototype.

23. CONCLUSION

The project successfully demonstrates how Natural Language Processing can be used to analyze emotional and sentimental information in text. The workflow combines preprocessing, tokenization, stop-word removal, lemmatization, emotion mapping, frequency counting, VADER sentiment analysis, and visualization. The supplied output graph confirms successful emotion-frequency visualization with happy, sad, fear, and anger categories. Overall, the project provides a clear foundation for learning and further developing NLP-based emotion and sentiment analysis systems.

24. REFERENCES

1. Natural Language Toolkit (NLTK) documentation and package resources.
2. VADER sentiment analysis implementation available through NLTK.
3. Matplotlib documentation for Python visualization.
4. Python standard library documentation for file handling, string processing, and Counter.

PROJECT DECLARATION

I, **KUMARI SAUMYA**, Registration No. **25MIM10186**, present this project report titled “**Emotion and Sentiment Analysis Using NLTK**” as documentation of the implemented NLP project. The report describes the project's objective, methodology, implementation, results, limitations, and future scope.

Student Name	KUMARI SAUMYA
Registration No.	25MIM10186
Signature	_____
Date	_____